PROMPT PARTY

MASTER PRODUCT AND INTEGRATION SPECIFICATION

Standalone Twitch-ready audience-controlled AI party-game platform

Packet	00
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Product Definition

Prompt Party is a standalone live game-show application in which six to eight AI participants perform as contestants, actors, artists, lawyers, rappers, witnesses, or roasters while two configurable AI commentator seats host, react, referee, and sometimes judge. Human viewers submit approved challenges, trigger controlled modifiers, and help decide outcomes.

> **NON-NEGOTIABLE SEPARATION:** Use Team Talk as a reference implementation and source of reusable patterns. Create a separate repository, service, port, configuration, database, storage path, generated-media directory, UI, and deployment. Do not overwrite or convert the serious Team Talk application.

2. Version-One Games

Module	Primary output	Audience role	Commentator role
Art Showdown	Generated images	Prompt, modifier, vote	Expectation banter, visual critique, scoring
Rap Battle	Written verses; optional audio	Topic, forced words, vote	Hype, critique, scoring
AI Court	Arguments, testimony, verdict	Case facts, evidence, jury vote	Host, legal color, referee
AI Improv	Short scenes	Suggestions, twists, best-moment vote	Host, rule enforcement, recap
Roast Battle	Playful roast exchanges	Approved target/style, vote	Color commentary, safety referee, scoring

3. Product Surfaces

Surface	Purpose	Must show	Must never show
Broadcast	OBS/Twitch presentation	Challenge, cast, phase, output, captions, voting, result	API keys, raw exceptions, private moderation, hidden prompts
Audience	Room-code browser experience	Join, submit, choose, vote, confirmation	Producer controls or unapproved content
Producer	Private control room	Approvals, phase controls, retries, overrides, provider health	Nothing is hidden from producer except secrets masked by default
Replay	Round record and exports	Prompt, outputs, comments, scores, votes, failures, recoveries	Secrets and disallowed raw provider data

The uploaded visual prototype already reflects these separate views and should be treated as the visual-direction reference, not as the final application architecture.

4. Shared 12-Phase Lifecycle

Phase	Function
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play

Phase	Function
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

AUTHORITY RULE: Only the producer or trusted controller advances official state. A model response, audience vote, media callback, or provider completion cannot advance the round by itself unless the current state explicitly permits an automatic completion transition.

5. Core Data Contracts

5.1 Participant

```
{
 "participant_id": "judge_01",
 "display_name": "Verdict",
 "provider": "openai_compatible",
 "model": "configured-model",
 "seat_type": "commentator",
 "capabilities": [
  "text_generation",
  "vision_input",
  "structured_output"
 ],
 "persona_id": "brutal_critic",
 "avatar": "/assets/avatars/verdict.webp",
 "enabled": true,
 "timeout_seconds": 45,
 "max_retries": 1
}
```

5.2 Event envelope

```
{
 "event_id": "evt_01J...",
 "event_type": "judge.commentary_created",
 "show_id": "show_01J...",
 "round_id": "round_01J...",
 "game_id": "art_showdown",
 "phase": "PRE_REVEAL_COMMENTARY",
```

```
 "actor_type": "ai",
 "actor_id": "judge_01",
 "timestamp": "2026-07-27T17:00:00Z",
 "public": true,
 "payload": {}
}
```

5.3 Controlled modifier

```
{
 "modifier_id": "genre_shift",
 "game_id": "improv",
 "display_name": "Genre Shift",
 "allowed_phases": [
  "CREATION_ACTIVE"
 ],
 "target_types": [
  "scene"
 ],
 "duration": "next_two_turns",
 "requires_producer_approval": true,
 "audience_selectable": true,
 "effect": {
  "type": "scene_constraint",
  "instruction": "{{approved_genre}}"
 },
 "fallback": {
  "on_invalid_phase": "reject"
 }
}
```

6. Game Module Interface

```
class GameModule(Protocol):
   manifest: GameManifest
   def allowed_actions(self, state: RoundState) -> list[str]: ...
   def validate_transition(self, old: str, new: str) -> None: ...
   def build_ai_request(self, action: GameAction, context: GameContext) -> AIRequest: ...
   def parse_ai_response(self, action: GameAction, raw: str) -> StructuredOutput: ...
   def apply_modifier(self, modifier: Modifier, state: RoundState) -> RoundState: ...
   def calculate_score(self, round_record: RoundRecord) -> ScoreResult: ...
   def public_state(self, round_record: RoundRecord) -> dict: ...
```

Manifest field	Required meaning
game_id	Stable module identifier
display_name	Public game title
version	Prompt/schema compatibility version

Manifest field	Required meaning
seat_requirements	Minimum/maximum seats and capabilities
phases	Shared and game-specific phase mapping
actions	Producer and audience actions by phase
prompts	Versioned templates
schemas	Validated model outputs
modifiers	Approved effect objects
scoring	Weighting and tie rules
timeouts	Per-operation limits
fallbacks	Failure behavior

7. Commentator System

Two commentator seats are reusable performers whose job mode changes by game. Personality is data; authority is determined by the active game module.

Mode	Function	Examples
Host	Introductions, transitions, calls to action	Set up challenge, recap phase
Commentator	Fill generation or deliberation gaps	Predictions, callbacks, rivalry
Judge	Submit structured opinion scores	Art, rap, roast
Referee	Enforce game rules	Improv yes-and, court objections
Instigator	Execute approved chaos-card performance	Temporary role swap or forced defense

- Producer chooses the base persona before the show.
- Audience may trigger approved temporary modifiers but cannot replace the commentator system prompt.

- Commentators remain in opinion, comedy, performance, and visible-evidence lanes.
- A judge lacking required media access must not claim to see or hear the output.
- Scoring influence is game-specific; the audience remains the main decision-maker in scored games.

8. Audience Safety and Input Control

1. Receive audience input into a pending queue.
2. Run automated screening and rate limits.
3. Show producer a readable moderation reason.
4. Allow approve, reject, edit, pin, or combine where the game permits.
5. Lock the approved input into an immutable round record.
6. Send only the approved and escaped content to model providers.

- No unrestricted target selection for roast content.
- No audience-provided system instructions.
- No deceptive impersonation of private people or unauthorized real-person voice cloning.
- AI Court cases and evidence are clearly fictional game content.
- Political persuasion, doxxing, threats, protected-class abuse, and sexual content involving minors are blocked.

9. Failure-as-Content Contract

Layer	Required behavior
Technical	Record provider, model, operation, timeout/error, retry count, and raw details privately.
Controller	Keep state valid and expose retry, replace, skip, fallback, or technical draw.
Broadcast	Show an honest themed failure state without raw exception details.
Performance	Optionally request a short in-character recovery only after the real failure is recorded.
Truth rule	Never claim a missing image, verse, audio file, testimony, or vote result exists.

10. API and Streaming Surface

```
POST /api/shows
POST /api/shows/{show_id}/start|pause|resume|end
POST /api/shows/{show_id}/rounds
```

```
POST /api/rounds/{round_id}/transition
POST /api/rounds/{round_id}/submissions
POST /api/submissions/{submission_id}/approve|reject
POST /api/rounds/{round_id}/actions/{action_id}
POST /api/rounds/{round_id}/votes/open|close
POST /api/rounds/{round_id}/votes
POST /api/rounds/{round_id}/winner/calculate|override
GET  /api/shows/{show_id}/broadcast-state
GET  /api/shows/{show_id}/audience-state
GET  /api/shows/{show_id}/producer-state
GET  /api/shows/{show_id}/events   # SSE preferred for v1
```

11. Persistence

Record	Minimum contents
shows	Title, status, selected games, producer settings, timestamps
rounds	Game, phase, prompt, cast, modifier, result
participants	Provider/model/capability/persona configuration
audience_members	Room session, display name, rate-limit data
submissions	Original text, moderation status, edits, final locked form
ai_requests/responses	Template version, model, timing, parse status, usage
media_assets	Type, path, checksum, dimensions/duration, moderation status
votes	Voter session, choice, timestamps, replacement history
events	Ordered public/private event ledger
errors	Technical details and recovery action
exports	Generated round or show artifacts

12. Security and Separation

- Store provider secrets in the separate Prompt Party configuration or secret store; never inside round records.
- Mask secrets in the Producer UI and logs.
- Use room-scoped audience tokens and server-side vote validation.
- Serve generated media through controlled URLs; validate MIME type and file existence before public reveal.
- Keep public/private event filtering on the server, not only in front-end code.
- Assign Prompt Party a separate system service and data directory from Team Talk.

13. Repository Shape

```
prompt-party/
  app/
    api/
    controller/
    audience/
    broadcast/
    producer/
    providers/
    moderation/
    persistence/
    games/
      shared/
      art_showdown/
      rap_battle/
      ai_court/
      improv/
      roast_battle/
  web/
    broadcast/
    audience/
    producer/
    replay/
  tests/
  config/
  media/
  exports/
```

14. Build Order

7. Fork or extract generic Team Talk provider and streaming patterns into the new repository.
8. Implement show, round, event, command-idempotency, and public/private state models.
9. Build the Producer and Broadcast shells from the supplied visual direction.
10. Complete Art Showdown end to end as the reference module.
11. Add audience room joining, submissions, and voting.
12. Implement Rap Battle, Court, Improv, and Roast modules against the same interface.
13. Add replay/export, recovery tests, and deployment scripts.

14. Run five consecutive rounds of each game without state corruption before calling version one complete.

15. Master Acceptance Gate

- Existing Team Talk deployment and storage remain untouched.
- All five modules register and load through one engine.
- Invalid transitions and duplicate commands are rejected.
- Audience inputs never bypass moderation and producer approval.
- Public views never expose secrets, raw exceptions, or private prompts.
- Every game survives one model timeout without restarting the show.
- Commentators react coherently using bounded recent context.
- Votes close deterministically and overrides are audited.
- Replay records preserve prompt, outputs, commentary, score, vote, failure, and recovery.
- OBS can capture the Broadcast View cleanly at 16:9.